CANVIA

REST ASSURED + CUCUMBER · SESIÓN 2 DE 4

Autenticación y ciclo de vida completo

Token con `POST /auth`, cambios con PUT, PATCH y DELETE, objetos Java en lugar de mapas y la forma de la respuesta validada con JSON Schema.

Instructor **Giancarlo Palomino** Miércoles 28 de octubre · 15:00 a 17:00 Entrega: **feature/s2-<tu-nombre>**

HOY · 120 MINUTOS

De consultar reservas a gobernar todo su ciclo

HORA	MIN	QUÉ HACEMOS
15:00 – 15:10	10	Revisión de los PR de ayer: lo que más se repitió
15:10 – 15:40	30	Token con `POST /auth`, cabeceras, `PUT`, `PATCH` y `DELETE`
15:40 – 16:00	20	Objetos Java en lugar de mapas: serialización con Jackson
16:00 – 16:20	20	Validar la forma de la respuesta con **JSON Schema**
16:20 – 17:00	40	**Ejercicios 3 y 4** · entrega por Pull Request

3

Ejercicio 3

Autenticarse y actualizar una reserva con token (y fallar sin él).

4

Ejercicio 4

Ciclo de vida: crear → consultar → actualizar → eliminar → comprobar que ya no existe.

PARTIMOS DE

Tu rama de ayer con los ejercicios 1 y 2 en verde, ya integrada en `develop`.

LO QUE MÁS SE REPITE

Cinco observaciones que valen para todos

1 · SOLO MIRA EL 200

Un escenario que verifica el código y nada más pasa aunque la API devuelva otra reserva. **Valida código y contenido.**

2 · IDS FIJOS

`consulto la reserva 1` depende de que nadie la borre. **Cada escenario crea sus datos.**

3 · PASOS TÉCNICOS EN EL .FEATURE

URLs, JSON o nombres de cabeceras en el Gherkin. **Eso vive en Java.**

4 · STEPS DUPLICADOS

Dos métodos que hacen lo mismo con frases distintas. Antes de escribir un step, busca si ya existe (`Ctrl` + `Shift` + `F`).

5 · COMMITS Y RAMAS

Commits en español (`feat: agrega búsqueda por apellido`) y la rama del sílabo: `feature/s1-<tu-nombre>`.

LO QUE ESTUVO BIEN

Escenarios en verde en la CI, Data Tables bien convertidas y tildes y eñes guardadas sin cambios.

01

Autenticación y cambios protegidos

Consultar es libre. Cambiar o borrar exige demostrar quién eres: un token.

POST /AUTH

Primero el token, después el cambio

1 · PEDIR
POST /auth
usuario y clave en el cuerpo

→

2 · RECIBIR
{ "token": "952844ac..." }
se guarda en el Contexto

→

3 · ENVIAR
Cookie: token=...
en cada PUT, PATCH o DELETE

→

4 · RESULTADO
200 o 403
con token / sin token

api/AutenticacionCliente.java — JAVA

```java
public Response autenticar(String usuario, String clave) {
    return given()
            .spec(Especificaciones.base())
            .body(new Credenciales(usuario, clave))   // record → JSON
        .when()
            .post("/auth");
}

/** Devuelve un token válido o falla con un mensaje claro. */
public String obtenerToken(String usuario, String clave) {
    return autenticar(usuario, clave)
        .then()
            .statusCode(200)
            .body("token", notNullValue())
            .extract().path("token");
}
```

steps/AutenticacionSteps.java — JAVA

```java
@Dado("que estoy autenticado como administrador")
public void autenticadoComoAdministrador() {
    contexto.token = autenticacion.obtenerToken(
        Configuracion.usuario(), Configuracion.clave());
}
```

DOS MÉTODOS, DOS USOS

`autenticar` devuelve la respuesta cruda para probar el login. `obtenerToken` es para los escenarios que solo necesitan el token y deben fallar rápido si no llega.

DÓNDE VIAJA EL TOKEN

Cookie, cabecera o nada: lo decide la API

api/ReservaCliente.java — JAVA

```java
public Response actualizar(int id, Reserva reserva, String token) {
    return conToken(token)
            .pathParam("id", id)
            .body(reserva)
            .when().put("/booking/{id}");
}

/** Restful Booker recibe el token en la cookie "token". Sin token, no hay cookie. */
private RequestSpecification conToken(String token) {
    RequestSpecification peticion =
given().spec(Especificaciones.base());
    if (token != null && !token.isBlank()) {
        peticion.cookie("token", token);
    }
    return peticion;
}
```

UN MÉTODO PARA TRES CASOS

Con token real, con token inventado y sin token: el mismo `conToken` sirve para las pruebas positivas y las negativas.

LA API PIDE	ASÍ SE VE EN HTTP
Cookie (Restful Booker)	`Cookie: token=952844ac…`
Bearer (Canvia Market, examen)	`Authorization: Bearer <token>`
Básica	`Authorization: Basic YWRtaW46…`

SIEMPRE EN LA DOCUMENTACIÓN

Restful Booker acepta la cookie o una cabecera `Authorization: Basic`. Lee el contrato antes de escribir el cliente: si el token va en el lugar equivocado, la API responde **403** aunque el token sea válido.

PREGUNTA PARA LA SALA

¿Qué diferencia hay entre **401** y **403**? ¿Cuál debería devolver una API si no envías token?

CAMBIAR Y BORRAR

PUT reemplaza, PATCH retoca, DELETE borra

VERBO	CUERPO	CON TOKEN	SIN TOKEN
PUT	La reserva **completa**	200	403
PATCH	Solo los campos que cambian	200	403
DELETE	Ninguno	201 [¡ojo!]	403

api/ReservaCliente.java — JAVA

```java
public Response actualizarParcial(int id, Map<String, Object> cambios,
String token) {
    return conToken(token)
            .pathParam("id", id)
            .body(cambios)
            .when().patch("/booking/{id}");
}

public Response eliminar(int id, String token) {
    return conToken(token)
            .pathParam("id", id)
            .when().delete("/booking/{id}");
}
```

PUT

Si olvidas un campo, la API lo borra o rechaza la petición. Por eso enviamos el objeto entero con el precio cambiado: `reservaEnviada.conPrecio(999)`.

PATCH

Basta un `Map.of("additionalneeds", "Cuna")`. La prueba debe comprobar que **el resto no cambió**.

DELETE

Verificar el código no basta: un `GET` posterior debe responder **404**. Eso es el ejercicio 4.

CREDENCIALES FUERA DEL CÓDIGO

Ni la URL ni la clave se escriben en Java

support/Configuracion.java — JAVA

```java
public static String usuario() {
    return valor("api.usuario", "API_USUARIO");
}

private static String valor(String propiedad, String variable) {
    String deEntorno = System.getenv(variable);
    if (deEntorno != null && !deEntorno.isBlank()) {
        return deEntorno;
    }
    String deSistema = System.getProperty(propiedad);
    if (deSistema != null && !deSistema.isBlank()) {
        return deSistema;
    }
    throw new IllegalStateException("Falta la configuración \"" +
propiedad
            + "\": usa -D" + propiedad + "=... o la variable de
entorno " + variable);
}
```

1º Variable de entorno
API_USUARIO · secretos de la CI

→

2º Propiedad -D
o el valor por defecto del pom

→

3º Error claro
dice qué falta y cómo darlo

```
# Otra clave solo para esta ejecución
PS> mvn test "-Dapi.usuario=admin" "-Dapi.clave=otraClave"
# O por variable de entorno (PowerShell)
PS> $env:API_CLAVE = "otraClave"; mvn test
```

¿Y EL POM.XML?

Trae como valor por defecto la clave **pública** del sandbox de Restful Booker, que está en su documentación. Con una API real, ese valor no existe: llega solo por variable de entorno o por un secreto de GitHub.

02

Objetos Java en lugar de mapas

Un `Map` acepta cualquier cosa, incluso errores de tipeo. Un objeto con tipos no.

UN RECORD POR RECURSO

La reserva como objeto: nombres en español, JSON intacto

modelo/Reserva.java — JAVA

```java
@JsonInclude(JsonInclude.Include.NON_NULL)       // no envía los campos nulos
@JsonIgnoreProperties(ignoreUnknown = true)      // si la API agrega campos, no rompemos
public record Reserva(
        @JsonProperty("firstname") String nombre,
        @JsonProperty("lastname") String apellido,
        @JsonProperty("totalprice") Integer precio,
        @JsonProperty("depositpaid") Boolean depositoPagado,
        @JsonProperty("bookingdates") Fechas fechas,
        @JsonProperty("additionalneeds") String necesidades) {

    /** Una reserva válida de ejemplo para el huésped indicado. */
    public static Reserva de(String nombre, String apellido, int precio) {
        return new Reserva(nombre, apellido, precio, true,
                new Fechas("2026-11-01", "2026-11-05"), "Desayuno");
    }

    public Reserva conPrecio(Integer nuevoPrecio) {
        return new Reserva(nombre, apellido, nuevoPrecio, depositoPagado, fechas, necesidades);
    }
}
```

RECORD (JAVA 16+)

Clase inmutable en una línea: constructor, *getters*, `equals` y `toString` incluidos. Ideal para datos que viajan.

@JSONPROPERTY

En Java se llama `nombre`; en el JSON viaja como `firstname`. El código se lee en español sin romper el contrato.

INTEGER, NO INT

Si la API devuelve `"totalprice": null`, un `int` lo convierte en `0` sin avisar. Un `Integer` conserva el `null` y la prueba lo ve.

MÉTODOS "CON…"

Como el record es inmutable, `conPrecio(999)` devuelve una copia con un solo cambio: perfecto para PUT.

IDA Y VUELTA

Objeto → JSON al enviar, JSON → objeto al recibir

ENVIAR
.body(reserva)
Jackson serializa el record

→

API
POST /booking
{ "bookingid": 1333, "booking": {...} }

→

RECIBIR
.extract().as(ReservaCreada.class)
Jackson deserializa el JSON

steps/ReservaSteps.java · crear — JAVA

```java
private void crearValida(Reserva reserva) {
    contexto.reservaEnviada = reserva;
    contexto.respuesta = reservas.crear(reserva);
    ReservaCreada creada = contexto.respuesta.then()
            .statusCode(200)
            .extract().as(ReservaCreada.class);    // JSON → record
    contexto.idReserva = creada.id();
    contexto.reservasCreadas.add(creada.id());
}
```

steps/ReservaSteps.java · comparar — JAVA

```java
@Entonces("la reserva conserva sus demás datos")
public void conservaDemasDatos() {
    contexto.respuesta.then().statusCode(200);
    // JSON → record; equals() compara campo a campo
    Reserva guardada = reservas.consultar(contexto.idReserva)
            .then().statusCode(200)
            .extract().as(Reserva.class);
    assertThat("La reserva guardada", guardada,
equalTo(contexto.reservaEnviada));
}
```

UNA LÍNEA QUE VALE POR SEIS

Comparar dos records compara **todos** los campos. Si uno difiere, el mensaje muestra ambos objetos completos con `toString`: *Expected: Reserva[nombre=Elena, …, necesidades=Cuna] but: was Reserva[…].*

CONTRATO

JSON Schema: validar la forma, no solo los valores

src/test/resources/schemas/reserva.json — JSON

```
{
  "$schema": "http://json-schema.org/draft-04/schema#",
  "title": "Reserva (GET /booking/{id})",
  "type": "object",
  "required": ["firstname", "lastname", "totalprice", "depositpaid",
"bookingdates"],
  "additionalProperties": false,
  "properties": {
    "firstname": { "type": "string", "minLength": 1 },
    "lastname": { "type": "string", "minLength": 1 },
    "totalprice": { "type": "integer", "minimum": 0 },
    "depositpaid": { "type": "boolean" },
    "bookingdates": {
      "type": "object",
      "required": ["checkin", "checkout"],
      "additionalProperties": false,
      "properties": {
        "checkin": { "type": "string", "pattern": "^\\d{4}-\\d{2}-
\\d{2}$" },
        "checkout": { "type": "string", "pattern": "^\\d{4}-\\d{2}-
\\d{2}$" }
      }
    },
    "additionalneeds": { "type": "string" }
  }
}
```

pom.xml — XML

```
<dependency>
  <groupId>io.rest-assured</groupId><artifactId>json-schema-
validator</artifactId>
  <version>${restassured.version}</version><scope>test</scope>
</dependency>
```

steps/RespuestaSteps.java — JAVA

```
@Entonces("la respuesta cumple el contrato {string}")
public void cumpleContrato(String contrato) {
    contexto.respuesta.then()
            .body(matchesJsonSchemaInClasspath("schemas/" +
contrato + ".json"));
}
```

MISMA VERSIÓN

`json-schema-validator` 6.3.0, igual que `rest-assured`: por eso usa `${restassured.version}`. Los esquemas viven en `src/test/resources/schemas/`.

ASERCIONES + ESQUEMA

Cada herramienta atrapa un tipo de error distinto

SI LA API UN DÍA…	ASERCIÓN BODY(…)	JSON SCHEMA
devuelve otro nombre	la atrapa	no la ve
cambia `totalprice` a texto `"150"`	solo si lo miras	la atrapa
deja de enviar `depositpaid`	solo si lo miras	la atrapa
agrega un campo no documentado	no la ve	la atrapa
cambia el formato de fecha	solo si lo miras	la atrapa

features/contrato.feature — GHERKIN

```
@smoke
Escenario: Una reserva consultada cumple el contrato y llega rápido
  Dado que existe una reserva a nombre de "Inés"
  Cuando consulto esa reserva
  Entonces el código de respuesta es 200
  Y la respuesta cumple el contrato "reserva"
  Y la respuesta llega en menos de 2 segundos
```

REGLA PRÁCTICA

Esquema para la **forma** (tipos, obligatorios, formatos). Aserciones para el **negocio** (este nombre, este precio).

LO HACEMOS JUNTOS

Guion de la demo: del token al contrato

1. **Probar el login a mano.** `POST /auth` con la clave correcta y con una incorrecta. ¿Qué código devuelve la incorrecta?
2. **Escenario del token:** *Cuando me autentico como administrador → recibo un token de acceso.*
3. **PUT sin token** → 403. Luego con `Dado que estoy autenticado` → 200 y precio nuevo.
4. **Del Map al record:** reemplazar el `Map.of(…)` del día 1 por `Reserva.de(…)` ; correr y ver que sigue verde.
5. **Esquema:** agregar la dependencia, crear `reserva.json` y el paso *cumple el contrato*.
6. **Romperlo a propósito:** exigir un campo `"email"` en `required` y leer el error.

```
PS> mvn test "-Dcucumber.filter.tags=@smoke"
Tests run: 31, Failures: 0, Errors: 0, Skipped: 28
[INFO] BUILD SUCCESS

# paso 6: "email" agregado a required en reserva.json
✗ Y la respuesta cumple el contrato "reserva"
  Response body doesn't match expectation.
  error: object has missing required properties (["email"])
```

QUÉ MIRAR EN LA DEMO

El mensaje del esquema dice **qué** propiedad falta. Así se ve un cambio de contrato el día que la API cambie sin avisar.

3
NIVEL

SEGURIDAD

Autenticarse y actualizar con token

20 min
Dificultad ●●●○○

OBJETIVO

Obtener un token con `POST /auth` y usarlo para modificar una reserva con `PUT`. Comprobar que sin token (o con uno inventado) no se puede.

CRITERIOS DE ACEPTACIÓN

1. Un paso *Dado que estoy autenticado como administrador* guarda el token en el Contexto.
2. Con token, `PUT /booking/{id}` cambia el precio y responde **200** con el valor nuevo.
3. Sin token, la misma petición responde **403**. Con un token inventado, también.
4. Usuario y clave no están escritos en el código: se leen de `System.getProperty` o variables de entorno.

Aprendes: autenticación por token, cookies (`.cookie("token", token)`), PUT vs PATCH, pruebas negativas de seguridad, no versionar credenciales.

Punto de partida — GHERKIN

```gherkin
@ejercicio3 @seguridad
Escenario: El administrador actualiza el precio
  Dado que estoy autenticado como administrador
  Y que existe una reserva a nombre de "Carlos"
  Cuando actualizo el precio de esa reserva a 999
  Entonces el código de respuesta es 200
  Y el precio de la reserva es 999

@ejercicio3 @seguridad
Esquema del escenario: Nadie más puede actualizar
  Dado que existe una reserva a nombre de "Carlos"
  Cuando actualizo el precio con el token "<token>"
  Entonces el código de respuesta es 403
  Ejemplos:
    | token           |
    |                 |
    | token-inventado |
```

PARA PENSAR

Prueba `POST /auth` con una clave incorrecta. ¿Qué código devuelve? ¿Es el que esperarías? Anótalo: lo usarás en el ejercicio 5.

4
NIVEL

FLUJO COMPLETO

El ciclo de vida de una reserva

20 min
Dificultad ●●●●○

OBJETIVO

Crear → modificar parcialmente → borrar → confirmar que ya no existe. Y además, **limpiar** lo que creas.

CRITERIOS DE ACEPTACIÓN

1. `PATCH` cambia solo `additionalneeds`; el resto de campos queda igual (verifícalo).
2. Tras `DELETE`, un `GET` a ese id responde **404**.
3. Un hook `@After("@limpieza")` borra las reservas que el escenario creó, aunque el escenario falle.
4. Separa los steps en dos clases (`ReservaSteps`, `AutenticacionSteps`) que comparten el mismo `Contexto`.

Aprendes: PATCH, hooks condicionados por tag, limpieza de datos de prueba, steps en varias clases, records con Jackson.

Punto de partida — GHERKIN

```
@ejercicio4 @limpieza
Escenario: Una reserva nace, cambia y desaparece
  Dado que estoy autenticado como administrador
  Y que existe una reserva a nombre de "Elena"
  Cuando cambio sus necesidades adicionales a "Cuna"
  Entonces la reserva conserva sus demás datos
  Cuando elimino esa reserva
  Entonces el código de respuesta es 201
  Y la reserva ya no existe
```

DISCUSIÓN OBLIGATORIA

Restful Booker responde **201 Created** al borrar. Tu prueba pasa… ¿pero es correcto? ¿Lo reportarías como bug o lo aceptarías como contrato? Escribe tu respuesta en el PR.

PISTA

Guarda en el Contexto la reserva que enviaste (un record) y compárala completa con la que devuelve el `GET`.

LEER EL ROJO

Errores típicos de hoy y cómo salir de ellos

`403 con token`

El token va mal puesto

Cookie con otro nombre (`Token` , `auth`) o token en una cabecera que la API no lee. Revisa el log: *Cookies: token=…*

`precio=0 sin aviso`

Tipo primitivo

Si la API manda `null` y el record usa `int` , Jackson pone `0` en silencio y el bug se esconde. Usa `Integer` y `Boolean` .

`UnrecognizedPropertyException`

La API trajo un campo nuevo

Falta `@JsonIgnoreProperties(ignoreUnknown = true)` , o el nombre en `@JsonProperty` está mal escrito.

`Schema to use cannot be null`

No encuentra el esquema

La ruta es relativa a `src/test/resources` : `schemas/reserva.json` , sin `/` inicial ni `src/…` .

`PATCH borra campos`

Se enviaron nulos

Si mandas un record a medio llenar sin `@JsonInclude(NON_NULL)` , viajan `null` . Para PATCH, un `Map` con lo que cambia.

`405 en el After`

Limpieza de algo ya borrado

El hook intenta borrar una reserva que el escenario ya eliminó. No es un fallo: la limpieza no valida la respuesta.

CANVIA

CIERRE Y TAREA

Hoy tu prueba ya crea, cambia y borra con permiso

Entrega

PR **feature/s2-<tu-nombre>** a `develop` con los ejercicios 3 y 4, la CI en verde y tu respuesta sobre el 201.

Para practicar

Agrega el esquema `reserva-creada.json` y valida también la respuesta de `POST /booking`.

Mañana

Código que escala: especificaciones reutilizables, un cliente por recurso, ambientes, reportes y la caza de bugs.

Documentación: **rest-assured.io** · **json-schema.org**